

## 93. Catastrophic Backtracking

Kuch regular expressions simple dikhte hain par **bahut der** chal sakte hain aur JS engine ko "hang" bhi kar sakte hain. Typical lakshan: regexp kabhi theek chalta hai, par kuch strings par 100% CPU khaa ke atak jaata hai. Browser script kill karne ko kehta hai. Server-side JS me isse server process hi hang ho sakta hai, jo aur bura hai.

### Example

Check karna hai ki string words `(\w+)` se bani hai jinke baad optional space `(\s?)` ho. Natural regexp: `^(\\w+\\s?)*$`

```
js
let regexp = /^(\\w+\\s?)*$/;

regexp.test("A good string");           // true
regexp.test("Bad characters: $@#");     // false
```

Kaam karta hai, par kuch strings par bahut der lagta hai:

```
js
let str = "An input string that takes a long time or even makes this regexp hang";
regexp.test(str);    // bahut der / hang
```

(V8 8.8+ me Chrome 88 isse effectively handle karta hai, Firefox hang hota hai.)

### Simplified example

Spaces hatao, `(\w)` ko `(\d)` kardo: `^(\\d+)*$`. Ye bhi hang hota hai:

```
js
let regexp = /^(\\d+)*$/;
let str = "012345678901234567890123456789z";    // end me z zaruri hai
```

Regexp artificial hai par slow hone ki wajah wahi hai.

`123456789z` par kya hota hai:

- `(\\d+)` greedy hai, saare digits `123456789` le leta hai. Phir `(\\.\\.\\.)*` ko aur digits nahi milte. Pattern ka agla `(\\$)` chahiye, par text me `(z)` hai. Match nahi.
- Backtrack: `(\\d+)` ek character kam leta hai (`12345678`). Ab `(*)` ek aur `(\\d+)` lagata hai (`9`). Phir `(\\$)` fail (`z`).

3. Aur backtrack: `(\d+)` 7 digits, phir `(89)` ya `(8)`, `(9)` alag alag...

4. Sab combinations try hote hain.

`(123456789)` ko numbers me todne ke  $2^{(n-1)}$  tarike hain ( $n = \text{length}$ ). `(n=9)` par 511, `(n=20)` par ~10 lakh (1048575), `(n=30)` par ~1 arab (1073741823). Sab try karna hi time lagne ki wajah hai.

## Words aur strings par wapas

Pehle example `^(\\w+\\s?)*$` me bhi yahi: ek word `(\\w+)` ek baar ya kai tukdon me lag sakta hai:

```
(input)
(inpu)(t)
(inp)(u)(t)
(in)(p)(ut)
```

Insaan ko dikhta hai ki end me `(!)` hai to match ho hi nahi sakta, par engine ko nahi pata, wo saare combinations try karta hai (spaces ke saath `(\\w+\\s)*` aur bina spaces `(\\w+)*` dono, kyunki space optional hai).

**Lazy mode `(\\w+?)` se bhi kuch nahi hota:** combinations ka order badalta hai, kul ginti nahi. Kuch engines me tricky tests hote hain jo sab combinations se bachate hain, par zyadatar me nahi.

## Fix kaise karein? Do tarike

### 1. Combinations kam karo (regex rewrite)

Space ko **mandatory** banao: `^(\\w+\\s)*\\w*$` (kitne bhi words jinke baad space, phir optionally ek aakhri word). Ye pehle ke barabar hai par tez chalta hai:

```
js
let regex = /^(\\w+\\s)*\\w*$/;
regex.test(str); // false, tez
```

Kyun theek hua? Pehle space optional hone se `(\\w+)*` ban jaata tha jisme ek hi word `(\\w+)` ki kai repetitions me toot sakta tha. Ab `(\\w+\\s)*` me `(input)` ko do repetitions me nahi todha ja sakta, kyunki space zaruri hai.

### 2. Backtracking roko

Rewrite hamesha aasan nahi hota aur rewritten regex aksar complex hota hai.

Asli samasya ye ki engine aise combinations try karta hai jo insaan ko obviously galat lagte hain. `(\\d+)*$` me insaan ko pata hai `(+)` ko backtrack nahi karna chahiye. `^(\\w+\\s?)*$` me `(\\w+)` ko poora word lena chahiye, use todne ki zarurat nahi.

Modern engines me **possessive quantifiers** hote hain (quantifier ke baad `(+)`: `(\d++)`) jo backtrack nahi karte, aur **atomic groups**. Par JavaScript me ye supported nahi hain. Par "lookahead transform" se emulate kar sakte hain.

### Lookahead se possessive `(+)`

Pattern: `(?=(\w+))\1` (`(\w)` ki jagah koi bhi pattern).

- Lookahead `(?=)` aage dekhta hai sabse lamba word `(\w+)` (current position se)
- Lookahead ke andar ka content engine yaad nahi rakhta, isliye `(\w+)` ko parentheses me lo
- Phir `(\1)` se pattern me use reference karo

Yaani aage dekho, agar word `(\w+)` hai to use poora `(\1)` ki tarah lo. Lookahead poora word ek saath dhundhta hai aur `(\1)` use poora jod deta hai, to uske andar backtracking ki gunjaish hi nahi.

```
js
"JavaScript".match(/\w+Script/);           // JavaScript  (\w+ Java tak back
"JavaScript".match(/(?=(\w+))\1Script/);    // null        (poora word le liya)
```

Pehla example is tarah likhein:

```
js
let regexp = /^(?=(\w+)\2\s?)*$/;
regexp.test("A good string"); // true
regexp.test("An input string that takes a long time or even makes this regex ha
```

`(\2)` isliye kyunki bahar bhi parentheses hain. Numbering se bachne ke liye naam do:

```
js
let regexp = /^(?=(?<word>\w+)\k<word>\s?)*$/;
```

### Summary

Ye samasya "catastrophic backtracking" hai. Do hal:

- Regexp rewrite karo taaki combinations kam hon.
- Backtracking roko (JS me lookahead transform se).